LimaGo — Asignación del conductor más cercano

Proyecto Integrador N.° 2 · Algoritmos y Estructura de Datos II · Informe técnico

1. Contexto y datos

LimaGo asigna conductores sobre la red vial real de Miraflores (Lima), descargada de OpenStreetMap (relación 1944770, 9,45 km²) y guardada en *datos/miraflores.graphml*: Red vial: 1368 intersecciones, 2670 tramos dirigidos (1833 de sentido único), 227.4 km de tramos. La flota es simulada (300 conductores por defecto, semilla 42) y siempre se ubica en intersecciones de la mayor componente fuertemente conexa (1 300 nodos), de modo que todo conductor puede llegar a todo punto. El flujo de asignación tiene dos etapas, como en los sistemas reales: (1) un Quadtree propio con un heap obtiene los k = 5 conductores disponibles más cercanos en línea recta (Haversine propio); (2) un único Dijkstra propio sobre el grafo invertido, desde el pasajero, da la distancia por calles de los k candidatos respetando los sentidos de circulación; se asigna el de menor ETA (velocidad media configurable, 25 km/h por defecto).

2. Diseño del Quadtree

Es un Quadtree de puntos por región (x = longitud, y = latitud) con las clases **NodoQuadtree** y **Quadtree**. Cada nodo es hoja (lista de entradas) o interno (exactamente cuatro hijos NO, NE, SO, SE) y guarda dos contadores de su subárbol: *total* y *disponibles*. Así la búsqueda descarta ramas sin conductores libres sin visitarlas y la eliminación sabe cuándo fusionar. Un diccionario *id → entrada* permite eliminar, mover y cambiar disponibilidad en O(h) sin recorrer el árbol. **Bordes:** los rectángulos son cerrados, pero cada punto va a un único hijo por la regla *lon ≥ centro → este, lat ≥ centro → norte*; ningún punto se duplica ni se pierde. **Subdivisión acotada:** una hoja se divide solo si supera la capacidad (8) y su profundidad es menor que 18; si todos sus puntos tienen coordenadas idénticas no se divide (dividir nunca los separaría) y actúa como *cubeta*. Al eliminar, un nodo interno con *total* ≤ capacidad se fusiona en hoja. **k vecinos exactos:** búsqueda *best-first* (Hjaltason y Samet, 1999) con un min-heap de nodos ordenados por una cota inferior de distancia y un max-heap de tamaño k con los mejores candidatos. La cota es rigurosa sobre la esfera: $R\cdot|\Delta\varphi|$ fuera del rango de latitudes y $R\cdot\arcsin(\cos\varphi\cdot\sin\Delta\lambda)$ (distancia a un meridiano completo) fuera del de longitudes; se toma el máximo. Cuando la menor cota pendiente supera la k-ésima distancia, ningún nodo restante puede mejorar el resultado y la búsqueda termina. La exactitud se verificó contra la búsqueda lineal en todas las consultas del benchmark y en las pruebas automatizadas.

3. Análisis de complejidad (n conductores, c capacidad, h altura)

Operación	Costo	Justificación
Haversine	O(1)	Número fijo de operaciones trigonométricas.
insertar	O(h) amortizado	Descenso por un camino raíz-hoja; la división reparte ≤ c+1 puntos. h ≤ profundidad máxima (18); con datos dispersos h ≈ $\log_4$ n.
eliminar	O(h + c)	Camino hasta la hoja vía diccionario; la fusión recoge ≤ c puntos.
actualizar_posicion	O(h + c)	Misma hoja: O(h); si cambia de hoja, eliminar + insertar.
cambiar_disponibilidad	O(h)	Ajusta el contador de los nodos del camino.
buscar_rectangulo	O(h + v + m)	v nodos que intersectan el rectángulo, m resultados; peor caso O(n).
buscar_k_cercanos	O((v + d) log v + d log k)	v nodos visitados y d distancias calculadas; con datos dispersos v, d ≈ O(log n + k); peor caso (todos coincidentes) O(n log k).
Búsqueda lineal (referencia)	O(n log k)	Calcula n distancias y mantiene un heap de tamaño k.
Dijkstra (heap, borrado perezoso)	O((V + E) log V)	Cada arista se relaja una vez; cada push/pop cuesta O(log V). Con parada temprana explora solo hasta asentar los objetivos.
A* (Reto A)	O((V + E) log V) peor caso	Igual que Dijkstra; la heurística reduce los nodos expandidos.
Despacho completo	O(log n + k log k + (V'+E') log V')	Quadtree + un Dijkstra inverso que se detiene al asentar los k nodos.
Húngaro (Reto C)	O(p² m)	p pasajeros, m conductores candidatos (p ≤ m).

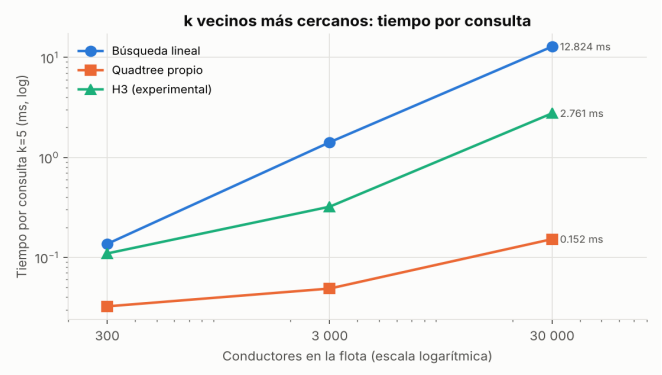
4. Metodología experimental

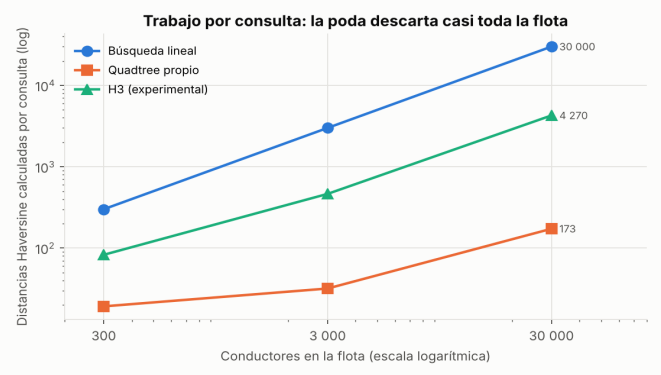
Equipo: x86_64, 4 núcleos lógicos, Linux-6.18.44-fc-v242-x86_64-with-glibc2.39, Python 3.13.18, networkx 3.7, osmnx 2.1.1 (fecha UTC 2026-11-06). **Búsqueda:** escalas exactas de 300, 3 000, 30 000 conductores; 400 pasajeros uniformes en el recuadro de la red (semilla 2026), k = 5, 20 % de ocupados, semilla de flota 42. Cada lote se repitió 5 veces con el recolector de basura desactivado y se reporta la mediana (robusta ante interrupciones del sistema operativo). Antes de medir se comparó cada respuesta con la búsqueda lineal. Escenario *intersecciones*: flota del RF-02 (muchos conductores comparten intersección); escenario *uniforme*: posiciones continuas al azar, para aislar el efecto de los puntos coincidentes. **Rutas:** 300 pares origen–destino (semilla 7); igualdad con networkx si $|\Delta| \le$ 1e-06 m: ambos suman los mismos *length* en float64 y solo puede variar el orden de la suma (error relativo del orden de 10^{-16}); una micra deja margen sin ocultar un tramo mal elegido, que cambiaría metros.

5. Resultados

Tabla 1. Tiempo mediano por consulta k = 5 (escenario intersecciones salvo la penúltima columna).

Conductores	Lineal (ms)	Quadtree (ms)	Aceleración	Distancias QT	Idénticos	Uniforme: QT (ms) / acel.	H3 (ms)
300	0,1365	0,0324	×4,2	19,2	400/400	0,0267 / ×5,1	0,1096
3 000	1,4175	0,0488	×29,0	31,8	400/400	0,0338 / ×32,6	0,3214
30 000	12,8238	0,1523	×84,2	172,5	400/400	0,0483 / ×252,4	2,7606

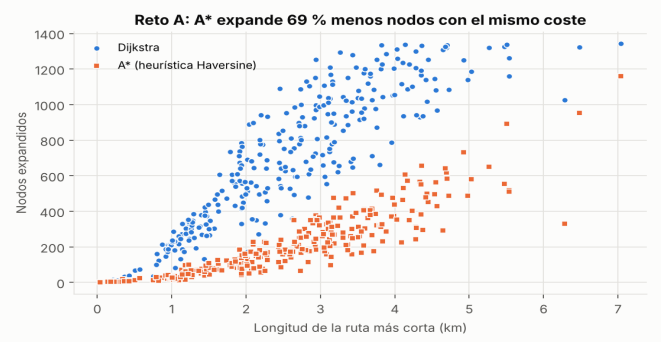


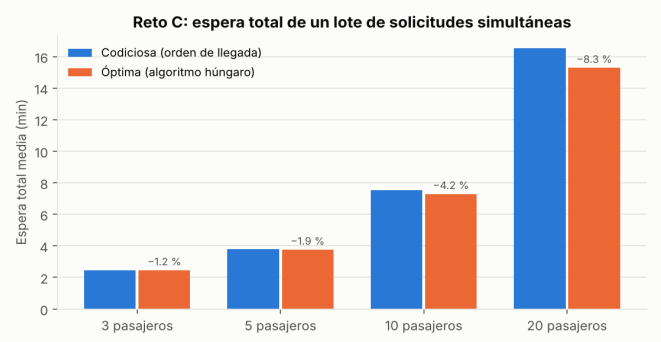


Interpretación. La búsqueda lineal crece proporcionalmente a n (×94 de 300 a 30 000), como predice O(n log k). El Quadtree crece mucho más despacio (×4,7 en el mismo rango): calcula en promedio 19, 32 y 173 distancias frente a 300, 3 000 y 30 000. Con 300 conductores la ventaja es solo ×4,2: la flota es tan pequeña que el costo de recorrer nodos y manejar dos heaps pesa casi tanto como recorrer la lista; el índice no es gratis y solo compensa al crecer n. En el escenario de intersecciones con 30 000 conductores la aceleración (×84) es menor que en el uniforme (×252) porque ~23 conductores comparten cada intersección: las cubetas coincidentes obligan a calcular todas sus distancias. No se observó ningún caso en que la lineal fuera más rápida, pero la brecha a 300 conductores es pequeña. **Dijkstra:** 300/300 distancias idénticas a networkx (diferencia máxima 0.0e+00 m; también en un árbol completo desde un origen). El Dijkstra propio con parada temprana tardó 0,467 ms por par y networkx 1,200 ms; la diferencia se explica porque networkx opera sobre el MultiDiGraph con diccionarios de atributos y nuestra lista de adyacencia ya trae el peso mínimo de cada par de nodos. El árbol completo: 1,18 ms (propio) frente a 2,64 ms. En el despacho, un Dijkstra inverso único costó 0,86 ms frente a 2,09 ms de k Dijkstra directos.

Mapas (RF-07). Se generaron 4 mapas folium en *mapas/*. En 3 de ellos el conductor asignado no es el más cercano en línea recta; p. ej., en «Parque Kennedy, Miraflores» el más cercano estaba a 110 m con ETA de 112 s, y el elegido a 228 m con ETA de 29 s.

6. Retos opcionales





A. A*. Heurística h(n) = Haversine(n, destino), admisible porque osmnx mide *length* como suma de arcos de círculo máximo con el mismo radio (6 371 009 m) y consistente por la desigualdad triangular esférica. Mismo coste que Dijkstra en los 300 pares y 208 nodos expandidos de media frente a 678 (−69,3 %). **B. Hora punta.** Congestión *simulada* (no hay datos de tráfico real): cada tramo cuesta longitud / velocidad libre del tipo de vía × factor horario con picos a las 8:00 y 18:30, más intenso en vías principales. Cambia los pesos del grafo: tiempo medio de 8,6 min a las 8:00 frente a 4,4 min a las 15:00; 215 de 300 rutas cambian de trazado y 9/100 asignaciones eligen otro conductor (mapa *mapas/reto_b_congestion_8_vs_15.html*). **C. Varios pasajeros.** Matriz de ETA (unión de los k candidatos de cada pasajero, un Dijkstra inverso por pasajero) resuelta con el algoritmo húngaro propio, validado contra fuerza bruta; un conductor nunca recibe dos solicitudes. Frente a la estrategia codiciosa por orden de llegada, la espera total baja 1,2 % con 3 pasajeros y 8,3 % con 20 (demanda concentrada, 40 ensayos): cuanto más compiten por los mismos conductores, más importa optimizar en conjunto. **D. H3.** Índice experimental aislado en *experimental/* (resolución 9, arista media ≈ 201 m) con búsqueda por anillos y una cota de parada conservadora. Dio resultados idénticos a la lineal (400/400), pero fue más lento que el Quadtree: 2,761 ms frente a 0,152 ms con 30 000 conductores, porque la resolución es fija (una celda

densa no se subdivide) y cada anillo exige llamadas a la librería.

7. Comparación con H3 de Uber y preguntas de investigación

¿Por qué hexágonos? En una malla hexagonal los seis vecinos de una celda comparten arista y sus centros están a la misma distancia; en una malla cuadrada hay ocho vecinos a dos distancias distintas (lado y diagonal). Por eso los anillos de hexágonos aproximan mejor un círculo, los suavizados y agregaciones por vecindad (oferta y demanda, precios dinámicos) no tienen sesgos direccionales y las celdas cubren el planeta con poca distorsión (H3 usa un icosaedro y necesita 12 pentágonos por resolución). El costo es que la jerarquía es aproximada: los 7 hijos no cubren exactamente al padre, a diferencia del Quadtree, que es exacto y adaptativo a la densidad. **¿Qué pasa si muchos conductores se concentran en un punto?** Un Quadtree ingenuo se subdividiría sin fin, porque dividir nunca separa puntos idénticos. Nuestra política lo evita: profundidad máxima y cubetas para coordenadas idénticas (probado con 500 conductores en el mismo punto). El costo se traslada a la búsqueda: toda la cubeta debe examinarse, lo que explica la menor aceleración del escenario de intersecciones. **¿Por qué el más cercano en línea recta no siempre llega primero?** Porque el auto circula por calles con sentido único, manzanas, vías sin retorno y velocidades distintas: Miraflores tiene 1 833 de 2 670 tramos de sentido único. Un conductor a una cuadra pero en contra del tránsito puede necesitar rodear la manzana. Por eso se toman k candidatos y se decide por la distancia real por calles (ETA).

8. Conclusiones y limitaciones

El índice espacial reduce el trabajo de cada búsqueda de O(n) a unas pocas decenas o cientos de distancias, con resultados exactos, y su ventaja crece con la flota; con flotas pequeñas la diferencia es modesta. La segunda etapa con Dijkstra propio es indispensable: el criterio de línea recta se equivoca con frecuencia en una red con muchos sentidos únicos. Limitaciones: la flota, la demanda y la congestión son simuladas; las posiciones se discretizan en intersecciones; elegir entre k candidatos no garantiza el ETA mínimo global; los tiempos dependen del equipo y deben reproducirse con *python -m benchmark.benchmark_busqueda*.

Referencias

Boeing, G. (2017). OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks. *Computers, Environment and Urban Systems*, 65, 126–139.

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1, 269–271.

Finkel, R. A. y Bentley, J. L. (1974). Quad trees: a data structure for retrieval on composite keys. *Acta Informatica*, 4, 1–9.

Hart, P. E., Nilsson, N. J. y Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Trans. Systems Science and Cybernetics*, 4(2), 100–107.

Hjaltason, G. R. y Samet, H. (1999). Distance browsing in spatial databases. *ACM Transactions on Database Systems*, 24(2), 265–318.

Kuhn, H. W. (1955). The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2, 83–97.

Samet, H. (1984). The quadtree and related hierarchical data structures. *ACM Computing Surveys*, 16(2), 187–260.

Sinnott, R. W. (1984). Virtues of the Haversine. *Sky and Telescope*, 68(2), 159.

Uber Technologies (2018). H3: Uber's Hexagonal Hierarchical Spatial Index. https://www.uber.com/blog/h3/ · Documentación: https://h3geo.org/docs/

Datos viales © colaboradores de OpenStreetMap, licencia ODbL. https://www.openstreetmap.org/copyright